System Verification and Validation Plan for Flexin'

Team 3, Flexin'
Arjun Karthik
Shao (Daniel) Hu
Noah Goldschmied
Peter Deforest
Daniel Franze-Da Silva

November 1, 2026

Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

[The intention of the VnV plan is to increase confidence in the software. However, this does not mean listing every verification and validation technique that has ever been devised. The VnV plan should also be a **feasible** plan. Execution of the plan should be possible with the time and team available. If the full plan cannot be completed during the time available, it can either be modified to "fake it", or a better solution is to add a section describing what work has been completed and what work is still planned for the future. —SS]

[The VnV plan is typically started after the requirements stage, but before the design stage. This means that the sections related to unit testing cannot initially be completed. The sections will be filled in after the design stage is complete. the final version of the VnV plan should have all sections filled in. —SS]

Contents

1 Symbols, Abbreviations, and Acronyms **iv**

2 General Information **1**
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 1
- 2.3 Custom Documents . . . 3
- 2.4 Relevant Documentation . . . 3

3 Plan **4**
- 3.1 Verification and Validation Team . . . 4
- 3.2 SRS Verification . . . 6
- 3.3 Design Verification . . . 6
- 3.4 Verification and Validation Plan Verification . . . 7
- 3.5 Implementation Verification . . . 7
- 3.6 Automated Testing and Verification Tools . . . 7
- 3.7 Software Validation . . . 8

4 System Tests **8**
- 4.1 Tests for Functional Requirements . . . 8
  - 4.1.1 Account Management . . . 8
  - 4.1.2 Fitness Tracking and Goals . . . 9
  - 4.1.3 Rewards and Virtual Currency . . . 9
  - 4.1.4 Simulated Investing . . . 9
  - 4.1.5 Community and Leaderboards . . . 12
  - 4.1.6 Progress Reporting . . . 14
- 4.2 Tests for Nonfunctional Requirements . . . 15
  - 4.2.1 Look and Feel . . . 15
  - 4.2.2 Usability and Humanity . . . 16
  - 4.2.3 Performance . . . 16
  - 4.2.4 Operational and Environmental . . . 17
  - 4.2.5 Maintainability and Support . . . 17
  - 4.2.6 Security . . . 18
  - 4.2.7 Cultural . . . 18
  - 4.2.8 Compliance . . . 18
  - 4.2.9 User Documentation and Training . . . 19
- 4.3 Traceability Between Test Cases and Requirements . . . 19

5 Unit Test Description **19**
5.1 Unit Testing Scope . . . 19
5.2 Tests for Functional Requirements . . . 20
5.2.1 Module 1 . . . 20
5.2.2 Module 2 . . . 21
5.3 Tests for Nonfunctional Requirements . . . 21
5.3.1 Module ? . . . 21
5.3.2 Module ? . . . 21
5.4 Traceability Between Test Cases and Modules . . . 22

6 Appendix **23**
6.1 Symbolic Parameters . . . 23
6.2 Usability Survey Questions? . . . 23

7 Appendix — Generative AI Use Disclosure **24**
7.1 AI Tools Used . . . 24
7.2 How and Where Used . . . 24
7.3 Verification of AI-Generated Content . . . 24

8 Appendix — Reflection **25**

List of Tables

1 Verification and Validation Team Roles . . . 5
[Remove this section if it isn't needed —SS]

List of Figures

[Remove this section if it isn't needed —SS]

1 Symbols, Abbreviations, and Acronyms

symbol	description
T	Test

[symbols, abbreviations, or acronyms — you can simply reference the SRS (**?**) tables, if appropriate —SS]

[Remove this section if it isn't needed —SS]

This document ... [provide an introductory blurb and roadmap of the Verification and Validation plan —SS]

2 General Information

2.1 Summary

The software being tested is Flexin', an app that allows users to learn about the power of compounding through working out and investing. The team will be looking to test the following functions of the app:

- Step tracking and integrations with fitness data sources (Android Health Connect and, optionally, Garmin watches)
- Real-time updates of fitness data within the app
- Converting fitness progress into virtual currency and depositing it into the user's account
- Viewing account balances
- Simulated/generated financial markets
- Performance leaderboards

2.2 Objectives

The main objective of the VnV plan is to ensure that Flexin' properly teaches the impact of compounding. For the VnV plan to achieve this objective, the team will prioritize the following qualities:

- **Learnability:** The app is easy for new users to learn, as measured by the learnability requirement in the SRS (UHR-EOU-08).
- **Correctness of currency calculations:** Virtual currency earned, deposited, and displayed in account balances is calculated correctly, and operations that change balances either complete fully or are rolled back (PR-RFT-02).

- **Consistency of asset calculations:** Simulated asset values are consistent and produce the returns expected from the implemented market algorithms, including the calibration against historical market data required by PR-PA-06.

- **User confidence in the financial lessons:** Users come away trusting and understanding the financial lessons the app is intended to convey.

The following objectives are out of scope for this VnV plan:

- **Assumptions in the SRS and Hazard Analysis:** The assumptions listed in the SRS (**?**) and the Critical Assumptions section of the Hazard Analysis are taken as given and will not be verified.

- **Third-party packages and libraries:** External dependencies, such as Android Health Connect and the Garmin Connect IQ SDK, are assumed to be correct and already verified by their implementation teams (e.g. Garmin). The team does not have the time or resources to verify these libraries within the scope of this capstone. This matches the Hazard Analysis assumption that third-party health platforms keep their documented behaviour. Flexin''s own handling of outages and malformed data from these sources is still in scope (PR-RFT-03).

- **Internet connectivity:** The SRS lists a mobile data or Wi-Fi connection as part of the environment needed to reach the Flexin' backend. The strength and reliability of the user's connection will not be tested. The app's required offline behaviour (PR-RFT-01 and PR-RFT-04) is still in scope.

- **Full verification of user-reported fitness activity:** The team will not attempt to fully verify that users are not lying about completing fitness goals, as this would require too much time and resources. The SRS already assumes that fitness devices report correct data and that artificial changes are dishonest use, and SC-04 limits this risk by not allowing manual entry of activity. Small checks may be used to gauge validity in some cases (e.g. checking that a Garmin watch's heart rate reading stays above a threshold for a period of time), but validity cannot be guaranteed, especially for users without a Garmin device who track activity solely through their phone.

2.3 Custom Documents

The team is completing two custom documents (CDs), one per term:

- **CD-Fall — User/Stakeholder/Client Interview Report:** A report on interviews with the project's client and potential users, used to elicit requirements. The SRS appendix already summarizes the two interviews conducted so far: a mock interview with the client and supervisor, and an interview with a representative end user.
- **CD-Winter — Business Plan:** A business plan exploring how Flexin' could be monetized after the capstone, as the team believes the project has the potential to become a startup. It will cover the market need, market size and business model, costs, competition, and an execution plan.

2.4 Relevant Documentation

The following documents are relevant to the VnV efforts:

- **Software Requirements Specification (SRS)** (**?**): Defines the functional and non-functional requirements of Flexin', each with a fit criterion. These fit criteria are what the tests in this plan check against. The SRS also lists the assumptions that bound what is in scope for verification, and names the testing tools the team will use (JUnit for backend unit and system tests, and Detox for end-to-end tests of the mobile client).
- **Hazard Analysis:** Identifies the hazards of using Flexin' and defines twelve safety and security requirements (SR1–SR12) to mitigate them, each with a fit criterion. Its roadmap commits to implementing SR1, SR2, SR3, SR6, SR8, SR9, SR11, and SR12 within the capstone, so these are the safety and security requirements this plan verifies. Its Critical Assumptions are excluded from verification.
- **Health Connect and Garmin Connect IQ SDK documentation:** Describes how activity data is provided to Flexin'. Since these platforms are assumed to keep their documented behaviour, their documentation defines the data the integration is expected to receive.

- **Custom Documents:** The interview report (CD-Fall) records the user and client needs that the requirements were derived from, which is what validation testing checks the app against. The business plan (CD-Winter) has little direct bearing on VnV, but one SRS scalability requirement refers to it for how the product might grow after the capstone.
- **User interviews and user testing:** The interviews in the SRS appendix ground the requirements in real user needs. The SRS also plans user testing on demo builds in Term 2 (Weeks 17 and 21), which will be used to assess learnability and user confidence in the financial lessons.

3 Plan

This section outlines the plan to verify and validate Flexin' across its project development lifecycle. The plan will detail the roles and responsibilities of the team, and will specify the approach to verify the SRS document, design and implementation, as well as how to verify the VnV itself. Lastly, it will describe the automated tools and software validation framework that the team plans on using.

3.1 Verification and Validation Team

The process of VnV will be a collective effort spread across the team and supervisor. All members will have defined roles that altogether will cover the process. These roles can be seen in Table 1.

Table 1: Verification and Validation Team Roles

Team Member	**Project Role**	**VnV Responsibilities**
Peter Deforest	Testing Lead	Coordinating all VnV activities, ensuring unit, integration and regression tests are written and maintained, and performing the final review of all test cases before each milestone.
Arjun Karthik	GitHub Manager, CI/CD Lead	Implementing and maintaining the CI/CD pipeline and static analysis tools, monitoring automated test coverage metrics, and resolving issues with VnV tooling.
Shao (Daniel) Hu	Lead Developer	Leading code inspections using a defined checklist, verifying that the design and implementation satisfy the SRS, and leading the code walkthrough for the supervisor.
Daniel Franze-Da Silva	Scribe, UI/UX Design Lead	Documenting VnV meeting minutes, preparing and running the usability survey, and verifying that the UI and UX meet the usability requirements in the SRS.
Noah Goldschmied	Team Liaison, Chairman	Maintaining traceability between requirements and test cases, coordinating SRS and VnV Plan reviews, and ensuring feedback from the supervisor, TAs and peer reviewers is addressed through GitHub issues.
Chris Schankula	Project Supervisor	Providing technical oversight and domain feedback on the fitness and finance features, reviewing the SRS and VnV Plan, and participating in the Revision 0 demonstration to validate the requirements.

3.2 SRS Verification

The SRS will be verified to ensure that all requirements detailed in the document are reasonable, accurate, understandable and complete. The bullet list below describes the different avenues in which the SRS may be reviewed.

- **Submission to TA:** The SRS will be submitted to our course TA, who will thoroughly read and analyze the document and provide feedback where needed.
- **Peer Review:** Classmates will review the SRS document and ensure its content makes sense from a course standpoint.
- **Presentation to Supervisor:** The team may meet with the project supervisor to ensure that the requirements described in the SRS are high quality and in line with what they expect to be delivered.

If any problem are found with the SRS, the team will utilize our issue board to keep track and fix them. Once development begins, these issues may also coincide with a bugfix.

3.3 Design Verification

Design verification will begin after the submission of the Design Document on November 16th. This verification will ensure that Flexin's architecture meets requirements and is implementable. The bullet list below describes how the Design Document may be reviewed.

- **Submission to TA:** The Design Document will be submitted to our course TA, who will thoroughly read and analyze the document and provide feedback where needed.
- **Peer Review:** Classmates will review the Design document and ensure its content makes sense from a course standpoint.
- **Presentation to Supervisor:** The team may meet with the project supervisor to ensure that the design is feasible from an implementation point of view.

Any problems with the Design Document will be handled through the teams issue board.

3.4 Verification and Validation Plan Verification

VnV plan verification will take place before the document is submitted by the team, as well as post submission by other members.

- **Team Review:** The team will review the VnV Plan ensuring that the defined tasks are feasible within the timeframe of the course and that the detail described in the document is appropriate.
- **Peer Review:** Classmates will review the VnV Plan and ensure its content makes sense from a course standpoint.
- **Presentation to Supervisor:** The team may meet with the project supervisor to ensure that the VnV Plan is feasible.
- **Submission to TA:** The VnV Plan will be submitted to the TA for grading and feedback.

3.5 Implementation Verification

Implementation verification ensures that the system, as built, correctly implements the SRS and design document. This will be done through a combination of the following:

- **Unit Testing and System Testing:** All functional and non-functional requirements will be verified through the tests described in the below sections of this document.
- **Static Analysis and Linting:** The team will enforce a coding standard using linters and static analyzers.
- **Code Review:** All code changes will require a pull request and approval from at least one team member. The team members will be looking for code correctness, standards, and proper error handling.

3.6 Automated Testing and Verification Tools

The team plans on creating standard unit tests that will run during the CI stage of the code development lifecycle. The framework used for the backend will most likely be JUnit, and for the frontend, Detox. The team may also create integration tests and end-to-end tests to ensure that application

functionality doesn't degrade in the grand scheme. Code coverage will be top of mind while developing this test plan, and a tool to ensure that a minimum agreed threshold of 80% coverage will be implemented.

3.7 Software Validation

The software validation step of the VnV plan will ensure that the near-final product lives up to its expectations. This validation will be done from the perspective of the users and the stakeholders.

- **Rev-0 Demonstration:** The Rev-0 demo will be used to ensure that the core functionality of Flexin' is up to the standards of what the supervisor expects.
- **Usability Tests:** The team plans on giving Flexin' to a select group of prospective users for a period of roughly 2 weeks. This will help the team understand what needs work, and what worked well for the users.

4 System Tests

[There should be text between all headings, even if it is just a roadmap of the contents of the subsections. —SS]

4.1 Tests for Functional Requirements

The subsections below group the functional requirements in the Functional Requirements section of the SRS (**?**) by feature area. Together they cover FUNC-01 to FUNC-16. FUNC-17 (Sponsored Reward Redemption) is a stretch goal in the SRS Waiting Room, so it is not tested.

[Each test uses the fields Control, Initial State, Input, Output, Test Case Derivation, and How test will be performed. —SS]

4.1.1 Account Management

These tests cover creating an account and signing in.

SRS ID	Requirement
FUNC-01	User Registration
FUNC-02	User Authentication

[Add the test cases for these requirements. —SS]

4.1.2 Fitness Tracking and Goals

These tests cover importing activity data from third-party providers and creating and tracking fitness goals.

SRS ID	Requirement
FUNC-03	Third-Party Fitness Data Import
FUNC-04	Fitness Goal Creation
FUNC-05	Fitness Goal Tracking

[Add the test cases for these requirements. —SS]

4.1.3 Rewards and Virtual Currency

These tests cover awarding virtual currency for activity and keeping each user's balance.

SRS ID	Requirement
FUNC-06	Reward Allocation
FUNC-07	Virtual Currency Tracking

[Add the test cases for these requirements. —SS]

4.1.4 Simulated Investing

These tests cover the portfolio and trading features of the simulated market.

SRS ID	Requirement
FUNC-08	Portfolio Creation
FUNC-09	Asset Purchase
FUNC-10	Asset Sale
FUNC-11	Transaction Recording
FUNC-12	Portfolio Performance Calculation

test-FR-09	**Asset Purchase**
Type	Dynamic, Automatic
Initial State	The user is logged in and has enough virtual currency to purchase the asset.
Input/Condition	The `UserID`, the `AssetID`, and a call to the buy action.
Output/Result	The cost of the asset is deducted from the user's currency balance, and the asset is added to the user's portfolio.
Covers	FUNC-09. Users can purchase available assets if they have sufficient virtual currency, so the balance must decrease by the asset's cost and the asset must appear in the portfolio.
How test will be performed	After the purchase of an asset, the balance is checked to confirm the currency was removed and the portfolio is checked to confirm the asset was added.

test-FR-10	**Asset Sale**
Type	Dynamic, Automatic
Initial State	The user is logged in and holds at least one previously purchased asset in their portfolio.
Input/Condition	The `UserID`, the `AssetID`, and a call to the sell action.
Output/Result	The sale value is added to the user's currency balance, and the asset is removed from the portfolio.
Covers	FUNC-10. Users can sell assets currently held in their portfolio, so the proceeds must be credited and the holding removed.
How test will be performed	After the sale of an asset, the balance is checked to confirm the currency was added and the portfolio is checked to confirm the asset was removed.

test-FR-11	**Transaction Recording**
Type	Dynamic, Manual
Initial State	The user is logged in and has a non-empty transaction history.
Input/Condition	The `UserID`, and a request to view previous market transactions.
Output/Result	A list of all transactions the user has made, including the type (buy or sell), asset, quantity, price and time.
Covers	FUNC-11, and also confirms the effects of FUNC-09 and FUNC-10. Every purchase and sale generates a transaction record the user can view at any time, so the list must match the trades that were made.
How test will be performed	After completing one or more transactions on an account, the user's transaction list is viewed and checked against the trades that were made.

test-FR-12	Portfolio Performance Calculation
Type	Dynamic, Automatic
Initial State	The user is logged in and has a portfolio containing at least one asset.
Input/Condition	The `UserID`, and a request to retrieve all assets in the portfolio and calculate performance.
Output/Result	The portfolio's current value and its percentage gain or loss. A graph of growth or loss over time is shown to the user.
Covers	FUNC-12. Users can view their current portfolio value and unrealized gains or losses, which helps them judge how well their investment strategies have performed.
How test will be performed	Through manual inspection and the use of financial calculators, the displayed portfolio value, percentage gain or loss, and growth graph are measured and verified.

[Add the test cases for these requirements. —SS]

4.1.5 Community and Leaderboards

These tests cover friends lists and public and private leaderboard groups.

SRS ID	Requirement
FUNC-13	Leaderboard Participation
FUNC-14	Friend Management
FUNC-15	Private Leaderboards

test-FR-13	**Leaderboard Participation**
Type	Dynamic, Manual
Initial State	The user is logged in, and a leaderboard with open spaces is available to join.
Input/Condition	The `UserID` and the `LeaderboardID` of a leaderboard with open spaces.
Output/Result	The user joins the intended leaderboard and appears in its member list.
Covers	FUNC-13. A user can join a group, so after the join request the user must be a member of the chosen leaderboard and able to compete with its other members.
How test will be performed	After the user attempts to join the leaderboard, check that the user's `UserID` is linked to that `LeaderboardID`.

test-FR-14	**Friend Management**
Type	Dynamic, Automatic
Initial State	The user is logged in and can search for other users by `UserID`.
Input/Condition	The `UserID` of another application user.
Output/Result	A friend request is sent to that user, and the invitee can respond to it.
Covers	FUNC-14. Users can add and remove friends, so a request must reach the invitee, and the result of their response (approved or denied) must be reflected in both users' friends lists. This gives users the social aspect of comparing progress with selected friends.
How test will be performed	After the user sends a friend request, check that both the requester and the invitee can see the request and, once the invitee responds, the result of the action (approved or denied).

test-FR-15	Private Leaderboard Creation
Type	Dynamic, Automatic
Initial State	The user is logged in.
Input/Condition	A valid `UserID` and a request to create a new leaderboard.
Output/Result	A new private leaderboard is created with a unique `LeaderboardID`.
Covers	FUNC-15. Users can create private competition groups, so a successful request must produce a new group with its own ID. The created group is also needed by the FR-13 tests, which require a leaderboard to join.
How test will be performed	After the user completes the steps to create a leaderboard, check that a valid `LeaderboardID` exists and that the creator is a member of the new leaderboard.

4.1.6 Progress Reporting

These tests cover the summaries of activity, goals, earnings, portfolio performance, and standings.

SRS ID	Requirement
FUNC-16	Progress Reporting

test-FR-16	Progress Reporting
Type	Dynamic, Automatic
Initial State	The user is logged in and has a trackable history of activity, goals, currency earnings and portfolio activity.
Input/Condition	A valid `UserID` and a request to view the progress data connected to that ID.
Output/Result	Current and historical summaries of the user's fitness activity, goal completion, virtual currency earnings, portfolio performance and leaderboard standings, each matching the data recorded for that user.
Covers	FUNC-16. Users can access current and historical progress information at any time, so each summary must match the stored data. It also verifies the combined effects of FUNC-03, FUNC-04, FUNC-05, FUNC-06, FUNC-07 and FUNC-12, whose data feed the report.
How test will be performed	After the `UserID` is submitted to retrieve account progress, check that the returned summaries include the user's activity, goal tracking, currency earnings, portfolio performance and leaderboard standings if applicable

4.2 Tests for Nonfunctional Requirements

The subsections below follow the nonfunctional requirement sections of the SRS (**?**). Each one lists the SRS subsections and requirement IDs that its tests cover.

[Each test uses the fields Type, Initial State, Input/Condition, Output/Result, and How test will be performed. Static tests, such as reviews, inspections, and walkthroughs, should instead say who is involved and how the review is run. —SS]

4.2.1 Look and Feel

These tests cover the Look and Feel Requirements section of the SRS.

Appearance Covers LF-APP-01 to LF-APP-05.
[Add the test cases for these requirements. —SS]

Style Covers LF-STY-01 to LF-STY-04.
[Add the test cases for these requirements. —SS]

4.2.2 Usability and Humanity

These tests cover the Usability and Humanity Requirements section of the SRS.

Ease of Use Covers UHR-EOU-01 to UHR-EOU-09.
[Add the test cases for these requirements. —SS]

Personalization and Internationalization Covers UHR-PER-01 to UHR-PER-04, UHR-INT-01.
[Add the test cases for these requirements. —SS]

Learning Covers UHR-LRN-01 to UHR-LRN-04.
[Add the test cases for these requirements. —SS]

Understandability and Politeness Covers UHR-UND-01 to UHR-UND-03, UHR-POL-01 to UHR-POL-05.
[Add the test cases for these requirements. —SS]

Accessibility Covers UHR-ACC-01 to UHR-ACC-06.
[Add the test cases for these requirements. —SS]

4.2.3 Performance

These tests cover the Performance Requirements section of the SRS.

Speed and Latency Covers PR-SL-01 to PR-SL-04.
[Add the test cases for these requirements. —SS]

Safety-Critical Covers PR-SC-01 to PR-SC-08.
[Add the test cases for these requirements. —SS]

Precision or Accuracy Covers PR-PA-01 to PR-PA-06.
[Add the test cases for these requirements. —SS]

Robustness or Fault-Tolerance Covers PR-RFT-01 to PR-RFT-04.
[Add the test cases for these requirements. —SS]

Capacity Covers PR-C-01, PR-C-02.
[Add the test cases for these requirements. —SS]

Scalability or Extensibility Covers PR-SE-01 to PR-SE-03.
[Add the test cases for these requirements. —SS]

Longevity Covers PR-L-01, PR-L-02.
[Add the test cases for these requirements. —SS]

4.2.4 Operational and Environmental

These tests cover the Operational and Environmental Requirements section of the SRS.

Productization Covers PROD-01, PROD-02.
[Add the test cases for these requirements. —SS]

4.2.5 Maintainability and Support

These tests cover the Maintainability and Support Requirements section of the SRS.

Maintenance Covers MAS-MNT-01 to MAS-MNT-05.
[Add the test cases for these requirements. —SS]

Supportability Covers MAS-SUP-01.
[Add the test cases for these requirements. —SS]

Adaptability Covers MAS-ADPT-01 to MAS-ADPT-03.
[Add the test cases for these requirements. —SS]

4.2.6 Security

These tests cover the Security Requirements section of the SRS.

Access Covers SEC-ACC-01 to SEC-ACC-05.
[Add the test cases for these requirements. —SS]

Integrity Covers SEC-INT-01 to SEC-INT-03.
[Add the test cases for these requirements. —SS]

Privacy Covers SEC-PRV-01 to SEC-PRV-03.
[Add the test cases for these requirements. —SS]

Audit Covers SEC-AUD-01, SEC-AUD-02.
[Add the test cases for these requirements. —SS]

Immunity Covers SEC-IMM-01 to SEC-IMM-03.
[Add the test cases for these requirements. —SS]

4.2.7 Cultural

These tests cover the Cultural Requirements section of the SRS.

Cultural Covers CUL-01, CUL-02.
[Add the test cases for these requirements. —SS]

4.2.8 Compliance

These tests cover the Compliance Requirements section of the SRS.

Legal Covers COMP-LEG-01 to COMP-LEG-07.
[Add the test cases for these requirements. —SS]

Standards Compliance Covers COMP-STD-01 to COMP-STD-07.
[Add the test cases for these requirements. —SS]

4.2.9 User Documentation and Training

These tests cover the User Documentation and Training Requirements section of the SRS.

User Documentation Covers DOC-01 to DOC-04.
[Add the test cases for these requirements. —SS]

Training Covers TRN-01 to TRN-03.
[Add the test cases for these requirements. —SS]

4.3 Traceability Between Test Cases and Requirements

[Provide a table that shows which test cases are supporting which requirements. —SS]

5 Unit Test Description

[This section should not be filled in until after the MIS (detailed design document) has been completed. —SS]

[Reference your MIS (detailed design document) and explain your overall philosophy for test case selection. —SS]

[To save space and time, it may be an option to provide less detail in this section. For the unit tests you can potentially layout your testing strategy here. That is, you can explain how tests will be selected for each module. For instance, your test building approach could be test cases for each access program, including one test for normal behaviour and as many tests as needed for edge cases. Rather than create the details of the input and output here, you could point to the unit testing code. For this to work, you code needs to be well-documented, with meaningful names for all of the tests. —SS]

5.1 Unit Testing Scope

[What modules are outside of the scope. If there are modules that are developed by someone else, then you would say here if you aren't planning on verifying them. There may also be modules that are part of your software, but have a lower priority for verification than others. If this is the case, explain your rationale for the ranking of module importance. —SS]

5.2 Tests for Functional Requirements

[Most of the verification will be through automated unit testing. If appropriate specific modules can be verified by a non-testing based technique. That can also be documented in this section. —SS]

5.2.1 Module 1

[Include a blurb here to explain why the subsections below cover the module. References to the MIS would be good. You will want tests from a black box perspective and from a white box perspective. Explain to the reader how the tests were selected. —SS]

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

2. test-id2

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

3. ...

5.2.2 Module 2

...

5.3 Tests for Nonfunctional Requirements

[If there is a module that needs to be independently assessed for performance, those test cases can go here. In some projects, planning for nonfunctional tests of units will not be that relevant. —SS]

[These tests may involve collecting performance data from previously mentioned functional tests. —SS]

5.3.1 Module ?

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input/Condition:

   Output/Result:

   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

   Output:

   How test will be performed:

5.3.2 Module ?

...

5.4 Traceability Between Test Cases and Modules

[Provide evidence that all modules have been considered. —SS]

6 Appendix

This is where you can place additional information.

6.1 Symbolic Parameters

The definition of the test cases will call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance.

6.2 Usability Survey Questions?

[This is a section that would be appropriate for some projects. —SS]

7 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

7.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

7.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

7.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

8 Appendix — Reflection

[This section is not required for CAS 741 —SS]

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?